Data Structures & Algorithms (CSC-214) **Semester: Spring 2025**
Assignment 1 **Due date: 13th March 2025, 1 AM**

Submission:
-You are required to use Visual Studio for the assignment. Combine all your work (solution folder) in one .zip file after performing "Clean Solution". Name the .zip file as ROLLNUMBER-A1.zip (e.g. F2024-2345-A1.zip).
-Submit zip file on Google classroom within given deadline.
-Failure to submit according to above format would result in deduction of 10% marks.
Deadline: Deadline to submit assignment is 13th March, 2025, 1 AM. No submission will be considered for grading outside Google classroom or after this deadline. Correct and timely submission of assignment is responsibility of every student; hence no relaxation will be given to anyone.
Plagiarism: 0 marks will be given if any part of the assignment is found plagiarized. A code is considered plagiarized if more than 20% code is not your own work.

Mode of Testing: There will be an in-class quiz on Thursday, March 13, to check your understanding of the assignment. Remember, if you don't submit the assignment on Google Classroom before the deadline, you'll get 0 on the quiz.

Question 1: ResizableArray Class
You are required to design and implement a class ResizableArray that models a resizable array with extended functionalities. This class should manage elements dynamically and support a range of operations, including adding, removing, accessing elements, and more.

Requirements
Implement the ResizableArray class with the following methods:

1. Add an Element: void add(int value): Add value to the end of the array. Resize the array if necessary.
2. Insert an Element at a Specific Position: bool insert(int index, int value): Insert value at the specified index. If the index is out of bounds, return false. If the index is equal to the current size, it behaves like the add method. Resize the array if necessary.
3. Remove an Element: bool remove(int index): Remove the element at the specified index. Return true if the removal was successful, and false if the index is out of bounds.
4. Update an Element: bool update(int index, int value): Update the element at the specified index with value. Return true if the update was successful, and false if the index is out of bounds.
5. Get an Element: int get(int index) const: Return the value at the specified index. If the index is out of bounds, return -1.

6. Size of the Array: int size() const: Return the number of elements currently in the array.
7. Capacity of the Array: int capacity() const: Return the current capacity of the internal array.
8. Clear the Array: void clear(): Remove all elements from the array and reset its size to 0.
9. Reverse the Array: void reverse(): Reverse the order of elements in the array.
10. Find an Element: int find(int value) const: Return the index of the first occurrence of value in the array. If value is not found, return -1.

Constraints

- The array should automatically resize itself to be twice its previous capacity when it becomes full.
- The array should shrink to half its capacity when the number of elements falls below one-fourth of its capacity.
- The array should be initially empty with a capacity of 1.
- Ensure that resizing and other operations are efficient to handle up to 10^3 elements.

Question 2: CustomerList Class

You are developing a simple tool to manage a list of customer IDs for a retail store. The list is not sorted, and you need to implement functionality to search for specific customer IDs, among other operations. Your task is to implement a class CustomerList that allows you to perform the required operations.

Requirements

Implement the CustomerList class with the following methods:

1. Add a Customer: void addCustomer(int customerId): Add customerId to the list.
2. Find a Customer: int findCustomer(int customerId) const: Use linear search to find the index of customerId in the list. Return the index if found; otherwise, return -1.
3. Remove a Customer: bool removeCustomer(int customerId): Remove customerId from the list if it exists. Return true if the removal was successful; otherwise, return false.
4. Print List: void printList() const: Print all customer IDs in the list.

Constraints

- The list of customer IDs can contain up to 10^4 IDs.
- The list is not necessarily sorted.
- Ensure that operations are efficient enough to handle the maximum constraints

Question 3: SortedCatalog Class

You are developing a utility for a small business to manage a sorted list of product SKUs (Stock Keeping Units). The list of SKUs is kept in ascending order. Your task is to implement a class SortedCatalog that allows you to perform the following operations:

Requirements

Implement the SortedCatalog class with the following methods:

1. Add a Product: void addProduct(int sku): Insert sku into the catalog. Ensure the list remains sorted.

2. Find a Product: int findProduct(int sku) const: Use binary search to find the index of sku in the sorted catalog. Return the index if found; otherwise, return -1.
3. Print Catalog: void printCatalog() const: Print all SKUs in ascending order.

Constraints

- The catalog can contain up to 10^4 SKUs.
- The catalog is maintained in ascending order.
- Ensure that operations are efficient enough to handle the maximum constraints.